

Taller 15: Capítulo 20

Taller Arcgispro

Kevin Reyes

Tabla de contenidos

1	Resumen sintáctico	1
1.1	Pregunta 1: Listar Feature Classes de Punto y Línea	1
1.2	Pregunta 2: Describir una capa y sus propiedades	2
1.3	Pregunta 3: Verificar y crear un Feature Dataset	2
1.4	Pregunta 4: Listar campos de una tabla y sus tipos	3
1.5	Pregunta 5: Contar registros de una capa sin cargar los datos	3
1.6	Pregunta 6: Crear una capa temporal en memoria	3
1.7	Pregunta 7: Seleccionar registros por atributo en la capa temporal	4
1.8	Pregunta 8: Exportar los objetos seleccionados a un Feature Class físico	4
1.9	Pregunta 9: Filtrar con una consulta SQL compuesta	4
1.10	Pregunta 10: Limpiar las capas temporales en memoria	5
1.11	Pregunta 11: Leer registros y controlar valores nulos con SearchCursor	5
1.12	Pregunta 12: Extraer valores únicos de una columna con cursor y set	6
1.13	Tabla resumen de operaciones	7

1. Resumen sintáctico

A continuación se muestra una guía organizada por pregunta, con el código de ArcPy para cada solución y una tabla final que resume las operaciones.

1.1. Pregunta 1: Listar Feature Classes de Punto y Línea

Concepto: `arcpy.ListFeatureClasses()` permite obtener clases de entidad según el tipo de geometría.

```
import arcpy

puntos = arcpy.ListFeatureClasses(feature_type="Point")
lineas = arcpy.ListFeatureClasses(feature_type="Line")

print("Clases de entidad de puntos:", puntos)
print("Clases de entidad de líneas:", lineas)
```

1.2. Pregunta 2: Describir una capa y sus propiedades

Concepto: `arcpy.Describe()` devuelve metadatos de la capa, incluido el sistema de referencia y el campo OID.

```
import arcpy

capa_puntos = "points_3857"
desc = arcpy.Describe(capa_puntos)

print(f"Nombre de la entidad: {desc.name}")
print(f"Tipo de Geometría: {desc.shapeType}")
print(f"Campo ID interno: {desc.OIDFieldName}")
print(f"Sistema de Coordenadas: {desc.spatialReference.name}")
print(f"Unidades lineales: {desc.spatialReference.linearUnitName}")
```

1.3. Pregunta 3: Verificar y crear un Feature Dataset

Concepto: Un Feature Dataset obliga a capas internas a compartir el mismo sistema de referencia espacial.

```
import arcpy
import os

gdb_path = r"C:\PIG\Test.gdb"
dataset_name = "autumn"
referencia_capa = os.path.join(gdb_path, "points_3857")
dataset_completo = os.path.join(gdb_path, dataset_name)

if arcpy.Exists(dataset_completo):
    print(f"El Feature Dataset '{dataset_name}' ya existe en la base de datos.")
else:
```

```
spatial_ref = arcpy.Describe(referencia_capa).spatialReference
arcpy.management.CreateFeatureDataset(gdb_path, dataset_name, spatial_reference=spatial_ref)
print(f"Feature Dataset '{dataset_name}' creado exitosamente.")
```

1.4. Pregunta 4: Listar campos de una tabla y sus tipos

Concepto: `arcpy.ListFields()` permite validar tipos de datos antes de consultas SQL o cálculos.

```
import arcpy

campos = arcpy.ListFields("points_3857")

print("Estructura de campos (Primeros 5):")
for campo in campos[:5]:
    print(f"Campo: {campo.name} | Tipo: {campo.type} | Longitud: {campo.length}")
```

1.5. Pregunta 5: Contar registros de una capa sin cargar los datos

Concepto: `arcpy.management.GetCount()` es rápido porque consulta directamente la tabla en la geodatabase.

```
import arcpy

resultado_conteo = arcpy.management.GetCount("points_3857")
total_filas = int(resultado_conteo.getOutput(0))

print(f"La capa 'points_3857' contiene un total de {total_filas} registros puntuales.")
```

1.6. Pregunta 6: Crear una capa temporal en memoria

Concepto: Las capas virtuales permiten aplicar filtros y selecciones sin modificar la clase de entidad física.

```
import arcpy

capa_entrada = "points_3857"
capa_temporal = "lyr_puntos"
```

```

if arcpy.Exists(capa_temporal):
    arcpy.management.Delete(capa_temporal)

arcpy.management.MakeFeatureLayer(capa_entrada, capa_temporal)
print(f"Capa temporal '{capa_temporal}' creada en la memoria de la sesión.")

```

1.7. Pregunta 7: Seleccionar registros por atributo en la capa temporal

Concepto: `SelectLayerByAttribute` funciona sobre capas en memoria para aplicar filtros SQL.

```

import arcpy

clausula_sql = "bird_name = 'Folkert'"
arcpy.management.SelectLayerByAttribute("lyr_puntos", "NEW_SELECTION", clausula_sql)
conteo_filtrado = arcpy.management.GetCount("lyr_puntos")
print(f"Registros encontrados para el ave Folkert: {int(conteo_filtrado.getOutput(0))}")

```

1.8. Pregunta 8: Exportar los objetos seleccionados a un Feature Class físico

Concepto: `CopyFeatures` guarda la selección actual como una nueva clase de entidad en la geodatabase.

```

import arcpy
import os

gdb_path = r"C:\PIG\Test.gdb"
salida_fisica = os.path.join(gdb_path, "points_folkert")
arcpy.management.CopyFeatures("lyr_puntos", salida_fisica)
print(f"Datos exportados físicamente a: {salida_fisica}")

```

1.9. Pregunta 9: Filtrar con una consulta SQL compuesta

Concepto: Los filtros SQL combinados permiten seleccionar registros que cumplen varias condiciones.

```

import arcpy

if arcpy.Exists("lyr_filtro_ambiental"):
    arcpy.management.Delete("lyr_filtro_ambiental")
arcpy.management.MakeFeatureLayer("points_3857", "lyr_filtro_ambiental")
sql_compuesta = "vw_mtss IS NOT NULL AND vg_mtss_gcd > 4"

try:
    arcpy.management.SelectLayerByAttribute("lyr_filtro_ambiental", "NEW_SELECTION", sql_compuesta)
    resultado_conteo = arcpy.management.GetCount("lyr_filtro_ambiental")
    total_puntos = int(resultado_conteo.getOutput(0))
    print("Éxito: Consulta SQL ejecutada correctamente.")
    print(f"Puntos que cumplen con el criterio de vuelo activo con datos de viento: {total_puntos}")
except arcpy.ExecuteError:
    print("Error de geoprocesamiento de ArcGIS:")
    print(arcpy.GetMessages(2))
except Exception as e:
    print(f"Error general del script: {str(e)}")

```

1.10. Pregunta 10: Limpiar las capas temporales en memoria

Concepto: Siempre es buena práctica eliminar capas temporales al finalizar el análisis.

```

import arcpy

capas_a_borrar = ["lyr_puntos", "lyr_filtro_ambiental"]
for capa in capas_a_borrar:
    if arcpy.Exists(capa):
        arcpy.management.Delete(capa)
        print(f"Capa '{capa}' removida con éxito.")

```

1.11. Pregunta 11: Leer registros y controlar valores nulos con SearchCursor

Concepto: `arcpy.da.SearchCursor` permite iterar filas de atributos y validar `None` para evitar errores al formatear valores numéricos.

```

import arcpy

# Asegurar que el entorno apunte a la base de datos del proyecto
arcpy.env.workspace = r"C:\PIG\Test.gdb"

```

```

capa = "points_3857"
campos_analisis = ["bird_name", "vg_mtss_gcd"]

print("Extrayendo registros mediante cursores indexados (con control de nulos):")

# Abrir el cursor de lectura estructurada del módulo de acceso a datos (da)
with arcpy.da.SearchCursor(capa, campos_analisis) as cursor:
    contador = 0
    for fila in cursor:
        nombre_ave = fila[0]
        velocidad = fila[1]

        # Validar si el registro de velocidad contiene un valor nulo (None)
        if velocidad is not None:
            print(f"Ave: {nombre_ave} | Velocidad de avance: {velocidad:.2f} m/s")
        else:
            print(f"Ave: {nombre_ave} | Velocidad de avance: [DATO NULO / SIN REGISTRO]")

        contador += 1
        if contador == 10:
            break

```

1.12. Pregunta 12: Extraer valores únicos de una columna con cursor y set

Concepto: El script necesita conocer qué aves existen en la base de datos para construir bucles de filtrado espacial. Combinar un `SearchCursor` con el tipo de datos nativo `set` de Python elimina automáticamente duplicados.

```

import arcpy

capa = "points_3857"
campo_identificador = "bird_name"

aves_unicas = set()

with arcpy.da.SearchCursor(capa, [campo_identificador]) as cursor:
    for fila in cursor:
        # fila[0] contiene el valor de la columna 'bird_name'
        if fila[0] is not None:
            aves_unicas.add(fila[0])

```

```
print("Lista consolidada de aves en el dataset:", list(aves_unicas))
```

1.13. Tabla resumen de operaciones

Pregunta	Acción	Sintaxis ArcPy	Descripción breve
1	Listar Feature Classes de puntos y líneas	<code>arcpy.ListFeatureClasses()</code> <code>arcpy.ListFeatureClasses(feature_type="Point")</code> <code>arcpy.ListFeatureClasses(feature_type="Line")</code>	listar las capas según el tipo de la geodatabase.
2	Describir una capa	<code>arcpy.Describe(capa_puntos)</code>	obtener metadatos clave de la capa, como geometría y CRS.
3	Crear Feature Dataset	<code>arcpy.management.CreateFeatureDataset(...)</code>	crear un nuevo escenario dentro de la geodatabase con el mismo CRS.
4	Listar campos de la capa	<code>arcpy.ListFields("poligonos")</code>	Revisar los nombres, tipos y longitudes de campos antes de análisis.
5	Contar registros	<code>arcpy.management.GetCount("poligonos")</code>	Contar el número de filas sin cargar la tabla en memoria.
6	Crear capa temporal en memoria	<code>arcpy.management.MakeFeatureLayer(...)</code>	Crear una capa temporal en memoria para aplicar filtros y selecciones.
7	Seleccionar por atributo	<code>arcpy.management.SelectLayerByAttribute(...)</code>	Aplicar un filtro sobre la capa temporal.
8	Exportar selección	<code>arcpy.management.CopyFeatures("seleccion", salida_fisica)</code>	Exportar la selección a una nueva clase de entidad física.
9	Filtrar con SQL compuesto	<code>arcpy.management.SelectLayerByAttribute(...)</code>	Aplicar un filtro para refinar la selección espacial.
10	Limpiar capas temporales	<code>arcpy.management.Delete(...)</code>	Eliminar los objetos temporales para liberar memoria y evitar conflictos.

Pregunta	Acción	Sintaxis ArcPy	Descripción breve
11	Leer registros con SearchCursor y manejar nulos	<code>arcpy.da.SearchCursor(capa, campos_analisis)</code>	Leer capa con validación de <code>None</code> para evitar errores al mostrar valores numéricos.
12	Extraer valores únicos de un campo	<code>set()</code> con <code>arcpy.da.SearchCursor(capa, [campo_identificador])</code>	Genera lista de capas de aves sin duplicados usando el tipo nativo <code>set</code> .